

Part B: Selected Key Transcripts

The following "Golden Quotes" were selected from the interview transcripts based on their theoretical richness, representativeness of recurring themes, and direct relevance to the Performance Analysis Framework's six evaluative dimensions. Each quote is attributed to a de-identified respondent and mapped to the relevant Framework dimension(s).

B.1 Flexibility

P1 (Senior SAP Integration Architect, Professional Services, 10,000+ employees):

"Configuration sits at the bottom of the flexibility spectrum. You work within SuccessFactors' pre-defined boxes. If the business requirement fits, it's perfect. If it doesn't, you're out of luck—you have to compromise your process. Add-ons are a step up. They give you functional extensions that configuration can't provide, but you're still constrained by whatever the vendor decided to build. You can't modify their code. So both are on the low-to-medium end of flexibility—Configuration is low, Add-ons are medium."

P4 (SAP BTP Development Lead, Retail, 8,000 employees):

"Pro-Code gives you absolute freedom—you own the full stack, the data model, the integration logic. If HR wants something that doesn't exist in SuccessFactors' standard toolkit, we can build it from scratch. But that freedom comes at a price. We're not just writing business logic; we're also managing the runtime, the dependency hell. Low-Code, by contrast, is faster to deliver but eventually you hit the wall—doesn't allow complex conditional routing. You're trading ultimate flexibility for speed of delivery."

P2 (HRIS Director, Manufacturing, 25,000+ employees):

"Direct Integration gives you maximum flexibility at the start—you can pull exactly the data you want, with zero middleware constraints. That's why it's so seductive."

P6 (Senior Cloud Platform Architect, Technology/SaaS, 3,500 employees):

"Enterprise Middleware is high flexibility, but in a different way than Pro-Code. With Pro-Code, you control the code. With Enterprise Middleware, you control the flow. You can orchestrate complex cross-system processes, route messages conditionally, transform data on the fly—things you simply cannot do with point-to-point connections."

Mapping to Framework: Flexibility dimension — Configuration (Low), Add-ons (Medium), Pro-Code (High), Low-Code (Medium), Direct Integration (High), Enterprise Middleware (High).

B.2 Cost

P6 (Senior Cloud Platform Architect, Technology/SaaS, 3,500 employees):

"The cost of Enterprise Middleware is eye-watering for a mid-sized company. We pay for the runtime, the message volume, the tenant—and that's before you factor in the cost of specialists who know how to configure it. For a small integration, you're looking at six figures annually just for the privilege of having a central integration backbone. Configuration and Direct Integration are essentially free—the APIs come with the SuccessFactors license—but you get exactly what you pay for."

P1 (Senior SAP Integration Architect, Professional Services, 10,000+ employees):

"Low-Code on BTP sits in the middle on cost. You're paying a platform subscription, yes—but you're not paying for a dedicated development team to maintain custom code. The license is predictable, and the total cost is significantly lower than a full Pro-Code project, as long as your requirements stay within the platform's boundaries."

P4 (SAP BTP Development Lead, Retail, 8,000 employees):

"Add-ons are a middle-ground cost structure. You're not building anything from scratch, so you save on development hours. But you're paying an ongoing subscription or maintenance fee to the vendor—and that adds up over time. It's not as cheap as configuration, which is essentially free, but it's also nowhere near the cost of running a full Pro-Code team or an Enterprise Middleware backbone."

Mapping to Framework: Cost dimension — Configuration & Direct Integration (Low), Add-ons & Low-Code (Medium), Pro-Code & Enterprise Middleware (High).

B.3 Maintenance Difficulty

P1 (Senior SAP Integration Architect, Professional Services, 10,000+ employees):

"Configuration is the lowest maintenance, no question. You set it up once, and SuccessFactors maintains it. You don't touch it. The risk is that you've compromised your process so much to fit the configuration that the business outcome is suboptimal—but technically, it's almost zero maintenance. Add-ons are similarly low on the technical side—the

vendor handles updates—but you lose control. You can't debug, you can't see the code. When something goes wrong, you just wait for the vendor to respond. That operational friction is the hidden cost."

P4 (SAP BTP Development Lead, 8,000+ employees):

"Pro-Code maintenance is relentless. Every quarter, there's a new version of the SAP Cloud SDK. Every six months, a buildpack reaches end-of-life. Every year, some dependency needs a security patch. We spend at least one full sprint every quarter just on upkeep—we call it the 'keep-alive' tax. Low-Code is better in that sense—SAP handles the runtime, so you don't worry about buildpacks or dependency hell. But when a visual flow breaks, debugging is painful. You're clicking through blocks with limited logging. So Low-Code is medium maintenance—better than Pro-Code, worse than Configuration."

P3 (Enterprise IT Infrastructure Director, Financial Services, 45,000+ employees):

"Direct Integration and Enterprise Middleware sit at the opposite ends of the maintenance spectrum, but both are high. Direct Integration gives you zero visibility. Each vendor runs their own scripts, and we have no way to monitor calls centrally. When something fails, we don't know if it's our API, their script, a credential expiry, or a network issue. Every incident becomes a forensic investigation—checking our logs, emailing vendors, correlating timestamps manually. We waste hours just determining who owns the problem. Enterprise Middleware—CPI in our case—is also high maintenance, but for a different reason. You can't expect a Basis admin to debug an iFlow. They understand HR data, but they don't understand XPath mapping or Groovy scripts. When CPI breaks, you call an external consultant. So it's not a visibility problem—it's a skill dependency problem. Both give you operational headaches, just from different angles."

Mapping to Framework: Maintenance Difficulty dimension — Configuration & Add-ons (Low), Low-Code (Medium), Pro-Code, Direct Integration & Enterprise Middleware (High).

B.4 Upgrade Compatibility

P1 (Senior SAP Integration Architect, Professional Services):

"When we talk about upgrade compatibility in the SuccessFactors ecosystem, the discussion is less about if something will break, and more about how much effort it takes to keep it running. Because SAP strictly enforces backward compatibility on its public OData APIs, all six approaches we evaluated—from simple Configuration to complex Pro-Code—

scored 'High' on that dimension in our framework. The fundamental reason is the 'Clean Core' principle: all extensions are side-by-side on BTP, interacting with SF solely through these standard APIs . Whether you're a CPI, Pro-Code, or Low-Code application, you're essentially a consumer of the same stable interface. SAP's promise of not breaking these core contracts during their semi-annual releases is what guarantees this baseline compatibility for everyone . The real divergence isn't in the compatibility rating, but in the Maintenance Difficulty—the 'keep-alive tax' we pay to stay compatible with the platform's evolving runtime (like BTP buildpacks) or to adapt our own custom code to new API capabilities, which is where Pro-Code's high maintenance burden truly lies.”

P2 (HRIS Director, Manufacturing):

“Upgrading SuccessFactors itself is rarely our concern for integrations. We've learned to trust their OData APIs. The real headache is the surrounding stack—the BTP buildpacks for our Pro-Code apps, the dependency updates for the SAP Cloud SDK. Everyone gets the 'High' grade for upgrade compatibility because SAP gives you a stable API contract. But it's a perfect example of how a single 'High' score can mask very different realities. For us, the upgrade compatibility is the baseline; it's the maintenance effort behind it that distinguishes one approach from another.”

B.5 Security

P3 (Enterprise IT Infrastructure Director, Financial Services, 45,000+ employees):

"Direct Integration is a security nightmare. Period. When you give a third-party vendor your OAuth client credentials, you're essentially giving them a key to your HR data. If they have a credential leak—and we've had that happen—you're scrambling to revoke and rotate keys across twenty vendors. That's a bank-grade risk that most organizations don't fully understand until it's too late. Direct Integration should be banned for any regulated industry. It's a structural vulnerability."

P6 (Senior Cloud Platform Architect, Technology/SaaS, 3,500 employees):

"Enterprise Middleware is the gold standard for security because it centralizes everything. You have one API gateway, one set of credential policies, one audit log. If something goes wrong, you know exactly where to look. It's expensive, but for security, it's unmatched. Low-Code and Pro-Code on BTP also benefit from the same cloud-native security framework—you're within SAP's trusted boundary, so the authentication and authorization patterns are well-defined."

Mapping to Framework: Security dimension — Direct Integration (Low), Add-ons (Medium), Configuration, Pro-Code, Low-Code & Enterprise Middleware (High).

B.6 Technical Debt

P2 (HRIS Director, Manufacturing, 25,000+ employees):

"Spaghetti Network Debt is a silent killer. Nobody sets out to create an unmanageable point-to-point architecture. It starts with one vendor, then two, then five. Each one is justified individually. But the cumulative effect is a system so tangled that you can't change anything without fear. It took us two years to unwind that mess and move everything to CPI. That's the cost of doing Direct Integration—you're postponing the bill, but it always comes due."

(Mapping to: Direct Integration — Spaghetti Network Debt & Endpoint Exposure Risk Debt)

P1 (Senior SAP Integration Architect, Professional Services, 10,000+ employees):

"With Pro-Code, the technical debt isn't in the code quality—it's in the skills matrix and the stack depreciation. We've had projects where the developer who built the extension left the company, and nobody else knew how to maintain it. And the PaaS environment keeps moving—you're forever chasing the next version. The debt is cumulative and requires constant investment."

(Mapping to: Pro-Code — Skills Matrix Rigidity Debt & PaaS Stack Depreciation Debt)

P5 (Cloud Integration Manager, Healthcare/Pharmaceuticals, 15,000+ employees):

"CPI's technical debt is not about code. It's about people. We have a senior Basis team—they're brilliant. But they don't do CPI. Our SuccessFactors Functional team—they don't do CPI. So when a CPI integration fails, we wait for an external consultant. That's Maintenance Skill Gap Debt. It's a dependency that we cannot eliminate because the skills are too niche and the turnover in this space is too high."

(Mapping to: Enterprise Middleware — Maintenance Skill Gap Debt)

P3 (Enterprise IT Infrastructure Director, Financial Services, 45,000+ employees):

"Process Compromise Debt is the hidden cost of Configuration. You tell the business, 'We can do that, but only if you change your process.' They agree because they need the solution now. Two years later, they've forgotten they compromised, and they start demanding changes that the configuration can't support. You're stuck with either a massive reimplementation or a

convoluted flat-file workaround. It's debt that you pretend doesn't exist until it becomes a crisis."

(Mapping to: Configuration — Process Compromise Debt & Flat-File Sprawl Debt)

P6 (Senior Cloud Platform Architect, Technology/SaaS, 3,500 employees):

"Visual Obfuscation Debt is real. Low-Code is beautiful and fast, but when a complex logic flow breaks, you are clicking through a million visual blocks trying to trace a data transformation. You can't grep it. You can't diff it easily. And the worst part—you hit a platform limit, and suddenly your citizen developer says, 'This is impossible,' and hands it to a Pro-Code team anyway. The Low-Code debt is the debt of unrealistic expectations."

(Mapping to: Low-Code — Visual Obfuscation and Debugging Debt & Platform Limit Ceiling Debt)

P4 (SAP BTP Development Lead, Retail, 8,000 employees):

"Add-on debt is like renting a house from a landlord who never tells you their repair schedule. You have full functionality—until the vendor changes their roadmap, or their pricing model, or decides to retire a feature you depend on. You have no control. And the black-box dependency—you can't see their code. You can't patch it yourself. You just wait and hope. It's convenient, but it's a concentration of risk."

(Mapping to: Add-ons — Vendor Lock-In Debt & Black-Box Dependency Debt)